

Claude Code (CC) 零基础入门到精通完整教程

适用对象：从未接触 CC 的小白，以及想系统补齐知识的老用户

字数：约 4900 字 | 全文无概括性描述，所有命令均为可直接执行的完整字符串

第一章 认知准备：CC 为什么强

1.1 LLM Loop（大模型循环）

CC 是一个以大模型为大脑的 agent 程序，运作方式如下：

1. 用户输入提示词
2. 程序把提示词 + 工具说明书一起发给大模型
3. 大模型决定下一步用什么工具，并输出工具调用指令
4. 程序按指令执行工具（读写文件、跑命令、调 API）
5. 把执行结果返回给大模型
6. 大模型基于结果决定下一步，循环 2-5 直到任务完成
7. 程序把最终结果返回给用户

理解这一循环，就理解了为什么 CC 能"自主行动"而不是"一问一答"。

1.2 Harness（脚手架工程）

Harness 指 agent 程序中除大模型之外的所有设计——上下文管理、工具调度、权限系统、回滚机制等。**同一个大模型，搭配不同 harness，效果差异巨大。**CC 是当前 harness 工程的集大成者，这是它优于普通 agent 的根本原因。

1.3 CC 能做什么

不止编程：写文案、查资料、整理数据、做表格、做报告、操作飞书、发公众号、调研社交媒体、生成图片等知识工作均可。

第二章 安装与模型配置

2.1 前置准备

1. 下载并安装 Cursor（或 Trae、Windsurf 等 agent IDE）
2. 新建本地文件夹 first-cc，用 Cursor 打开

3. 调出 Cursor 内置终端

2.2 安装 CC（两选一）

方式 A：官方一行命令

- 1. 复制 CC 官网安装命令，粘贴到 Cursor 终端回车
- 2. 输入 `claude --version`，出现版本号即成功

方式 B：让 IDE Agent 帮装

- 1. 在 Cursor agent 框输入：帮我安装 Claude Code，处理依赖与网络问题
- 2. 等待自动完成，最后用 `claude --version` 验证

2.3 配置模型 API（CC switch）

- 1. 下载并安装 CC switch 工具
- 2. 必须在启动 CC 之前完成配置，否则 CC 默认走官方登录
- 3. 在 CC switch 中填写：API Key、Base URL、自定义模型名（高/中/低三档）
- 4. 保存后即可在 CC 内通过 `/model` 切换

2.4 启动 CC

- 1. 终端输入 `claude` 回车
- 2. 选择主题色 → 安全提示选 `yes` → 文件夹可信选 `yes`
- 3. 进入交互界面

2.5 全自动权限启动（无任何确认）

启动时附加参数：

```
claude --dangerously-skip-permissions
```

后续在终端按上方向键可快速复用该命令。

第三章 三种权限模式

模式	文件修改	终端命令	进入方式
计划模式 Plan Mode	先出方案再执行	先出方案再执行	默认进入或 <code>shift+tab</code> 切换

模式	文件修改	终端命令	进入方式
默认模式 Default	智能判断，需要时询问	需要时询问	shift+tab 切换
自动编辑模式 Auto-Edit	直接修改不询问	仍需询问	shift+tab 切换

切换快捷键： shift+tab ，三种模式循环切换。

计划模式下的四个选项：

- 1. 同意方案 + 切自动编辑模式
- 2. 同意方案 + 保持默认模式
- 3. 不同意，继续讨论
- 4. 按 shift+tab 提交反馈意见

终端命令确认时的三个选项：

- 1. yes ： 仅本次同意
- 2. 本项目后续同类命令不再询问
- 3. 拒绝并重新讨论

第四章 四种核心交互方式

4.1 文字指令

直接输入自然语言。换行键：

- macOS： option + 回车
- Windows： Ctrl + 回车
- ⚠ 不要按 shift + 回车 ，会直接发送

指令越具体越省 token。短指令会让 CC 花大量 token 通过 grep 探索本地文件来反推上下文。

4.2 @文件 / @文件夹

输入 @文件名 精准注入上下文，例如：

```
@main.js 解释代码逻辑并加中文详细注释
```

长提示词建议先写到本地 .md 文档里，再用 @文档名 引用，避免在终端里编辑长文本。

4.3 图片

- 拖拽图片到对话框
- 或 `Ctrl + V` 粘贴（macOS 也是 `Ctrl+V`，不是 `Cmd+V`，这是反直觉点）

适用于配色参考、设计稿还原等多模态需求。

4.4 Bash Mode (! 命令)

在对话框最前面输入 `!`，进入系统命令模式，直接执行电脑命令而非与 CC 对话：

```
!npm start
!ls -la
!git status
```

执行长进程后，按 `Ctrl + B` 让进程转入后台，避免阻塞 CC 对话。

第五章 斜杠命令速览

命令	作用
/help	列出所有命令
/model	切换高/中/低档模型
/BTW	提一个与当前项目无关的问题，回答完按 <code>ESC</code> 退出， 对话不污染上下文
/simplify	派生 3 个 sub-agent 从代码质量、运行效率、可复用性三角度审核优化
/rewind	打开回滚界面
/init	初始化项目，自动生成项目级 <code>claude.md</code>
/memory	管理 <code>claude.md</code> 与自动记忆
/context	查看上下文 token 详情
/compact	压缩上下文
/clear	清空上下文
/resolve	恢复历史对话
/agent	创建 sub-agent
/plugin	插件管理

第六章 回滚与版本管理

6.1 CC 自带快速回滚

- 快捷键：连按 **两次 ESC**
- 命令： `/rewind`
- 选项：仅回滚对话 / 回滚对话+文件 / 仅回滚文件 / 取消
-  **局限**：只能撤销 CC 编辑的文件，无法撤销已执行的终端命令（如 npm 安装）

6.2 Git 版本管理（真正的后悔药）

让 CC 用自然语言完成全流程：

1. 帮我安装 Git 并绑定我的 GitHub 账号
2. 把当前项目初始化为 Git 仓库并提交首个版本
3. 把当前分支推送到 GitHub 远程仓库
4. 帮我回滚到上一个稳定版本

习惯建议：每完成一个小功能，让 CC 存一次档，再尝试新方案就敢放手了。

第七章 上下文管理

7.1 为什么要管

大模型上下文有效比例只有 60%–80%，塞太满会变慢、变笨、抓不住重点。

7.2 三个核心命令

<code>/compact</code>	主动压缩，保留关键信息
<code>/clear</code>	彻底清空，等同新开对话
<code>/context</code>	查看 MCP / Skill / 历史对话各自占的 token

CC 在上下文快满时会自动压缩，但**任务切换时建议手动** `/compact`，让模型更专注新任务。

7.3 实时显示上下文占比

在 CC 中执行：

让 **CC** 帮我开启 `statusline`，显示上下文占比

重启终端后，底部会一直显示百分比。**经验阈值**：超过 60% 就该 `/compact`。

7.4 恢复历史对话

- 进 CC 后输入 `/resolve` 选要恢复的会话
- 或启动时直接 `claude -c` (continue 上次对话)

第八章 三层记忆机制

8.1 第一优先级：claude.md（强制注入）

三层叠加生效：

层级	路径	作用范围	适合写什么
全局	<code>~/.claude/CLAUDE.md</code>	所有项目	个人偏好（如"永远说中文"）
项目级	项目根目录 <code>CLAUDE.md</code>	当前项目	技术栈、架构、规范
文件夹级	子文件夹 <code>CLAUDE.md</code>	该文件夹内	局部特殊规则

生成方式：

- 项目级：项目有雏形后，运行 `/init`，CC 自动扫描并生成
- 编辑：`/memory` → 选层级 → 回车打开

Karpathy 案例：他的著名 skill 仅几百行通用规则，已 5 万 GitHub stars——claude.md 不是越长越好，应只放顶层不变原则。

8.2 第二优先级：Auto Memory（自动记忆）

开启：`/memory` → Auto Memory 选项 → 切到 on

记录范围：用户偏好 / 反馈纠错 / 项目决策 / 外部资源索引

存储与加载：

- 存放在 `.claude/projects/<项目>/memory/` 下
- 仅 `MEMORY.md` 索引文件被默认加载
- 详细内容按场景**按需加载**，不占满上下文

纠错：发送 忘掉刚刚关于 xx 的记忆，CC 自动删除对应文件与索引

注意：自动记忆仅作用于当前项目，换项目需重新积累。

8.3 第三优先级：自定义规范文档

自己写专项 `.md` 文档（如 `品牌视觉规范.md`、`文案语气规范.md`），然后在项目级 `claude.md` 里写：

当修改前端视觉时，参考 品牌视觉规范.md
当撰写产品文案时，参考 文案语气规范.md

CC 会按规则在合适时机加载，等同于给它一个**条件触发的外置大脑**。

第九章 高级拓展功能

9.1 Skill（技能）

四大分类 + 实例：

类型	例子
知识型	页面设计规范手册、品牌视觉指南
流程型	公司财务记账报销流程、合同审批 SOP
工具型	调用 nano banana 生成图片、调用 ffmpeg 压缩视频
混合型	公众号自动排版+发布、调研报告生成+推送

安装：

- 全局：放到 ~/.claude/skills/ 下
- 项目级：放到 项目根/.claude/skills/ 下
- 智能安装：先装 find-skill ，再让 CC 自动找并安装

调用：

- 自动：CC 看到提示词关键词自主调用
- 手动： /技能名 或在提示词里明确写"使用 XX 技能"
- ⚠ 当 Skill 多 + 上下文紧张时，触发率会下降，建议在提示词中明确提及技能名

自建 Skill：用 Anthropic 官方 skill-creator 通过对话生成。

9.2 MCP 与 CLI（外部工具连接）

MCP：AI 与外部服务的转接头（Notion、Figma 等），但**单个 MCP 占用 token 较多**，不适合堆量。

演进趋势：

- 轻量服务 → 转向 Skill
- 重量服务 → 转向 CLI

CLI：命令行工具，让 CC 用一行命令操作外部服务。

工具	用途
飞书 CLI	创建文档、多维表格、邮件、日历
OpenSale	抓取小红书、微博等社交媒体内容
GitHub CLI	仓库、PR、Issue 操作

安装：把 CLI 仓库链接发给 CC，让它自动安装并配置。

9.3 Sub-Agent（子代理）

作用：隔离上下文 + 并行执行，主 agent 只接收最终结论，避免被调研细节污染。

创建：

- 自动派生：CC 判断任务复杂时主动派生
- 手动： `/agent` ，用自然语言描述子 agent 的职责与权限

典型场景：让多个子 agent 并行调研竞品番茄钟，主 agent 继续写代码。

9.4 Hook（钩子，条件触发器）

作用：预设条件 → 自动执行命令。

示例配置指令：

```
帮我配置 Hook：每次 CC 完成任务后播放提示音并发送系统推送
帮我配置 Hook：每次提交代码前自动运行 ESLint 检查
```

按 CC 引导完成，不需要手写配置文件。

9.5 Plugin（插件）

插件 = Skill / Sub-Agent / Hook / MCP 的打包整合。

操作：

```
/plugin
```

进入管理页 → Discover 发现 → 选安装范围（全局/项目）→ 确认安装。

第十章 实操案例：30 分钟做一个 Electron 番茄钟

- 1. 启动 CC： `claude --dangerously-skip-permissions`
- 2. 输入： 做一个桌面番茄钟软件，技术栈用 Electron
- 3. CC 反问需求 → 选第一个预设方案
- 4. 进入计划模式 → 查看方案 → 选"同意方案+切自动编辑模式"
- 5. CC 创建文件 → npm 安装时选"本项目不再询问"
- 6. 运行：新开终端 `npm start` ，或原终端 `!npm start`
- 7. 转后台： `Ctrl + B`
- 8. 拖一张配色参考图进对话框，输入"按这张图改主题色"
- 9. 输入： `/simplify` 让 CC 三角度审核优化代码
- 10. 输入： 帮我提交当前版本到 Git，并推送到 GitHub

第十一章 新手避坑速记

- 1. 指令越具体，CC 烧的 token 越少、执行越准
- 2. 上下文超 60% 立即 `/compact`
- 3. 重要项目必须 Git 存档，CC 自带回滚撤不了终端命令
- 4. 自动记忆只在当前项目生效，换项目需重新积累
- 5. Skill 多时建议在提示词中点名调用，避免漏触发
- 6. macOS 粘贴图片用 `Ctrl+V` 而非 `Cmd+V`
- 7. 短指令未必省事，详细描述往往更省 token

附录 A 命令速查表

A.1 启动与会话

命令	作用
<code>claude</code>	启动 CC
<code>claude -c</code>	继续上次对话
<code>claude --dangerously-skip-permissions</code>	全自动权限启动
<code>claude --version</code>	查看版本

A.2 模式与交互

操作	作用
shift + tab	切换三种权限模式
option/Ctrl + 回车	换行（Mac/Win）
@文件名	注入文件上下文
Ctrl + V	粘贴图片
!命令	进入 bash mode 直接跑系统命令
Ctrl + B	长进程转后台
连按两次 ESC	快速回滚

A.3 斜杠命令

命令	作用
/help	命令帮助
/model	切换模型
/BTW	提无关问题，不污染上下文
/simplify	代码三角度审核
/rewind	回滚
/init	初始化项目并生成 claude.md
/memory	管理记忆
/context	查看上下文占比
/compact	压缩上下文
/clear	清空上下文
/resolve	恢复历史对话
/agent	创建子 agent
/plugin	插件管理

附录 B 学习自检清单

- [] 我能用一段话解释 LLM Loop
- [] 我知道 Harness 是什么，以及为什么 CC 比同类 agent 强

- ☐ 我能自由切换三种权限模式并说出区别
- ☐ 我会用 4 种交互方式（文字 / @文件 / 图片 / ! 命令）
- ☐ 我能熟练使用 `/compact` `/clear` `/context` 管理上下文
- ☐ 我会用 CC 完成 Git 安装、提交、推送、回滚全流程
- ☐ 我搭建过自己的 claude.md 三层记忆
- ☐ 我安装并调用过至少一个 Skill
- ☐ 我创建过至少一个 sub-agent
- ☐ 我用 CC 独立做出过一个完整小项目

完成 8 项以上，你已超越 90% 的 CC 用户。

本教程基于秋芝2046 《60分钟全面掌握 Claude Code》原文整理，感谢原作者贡献。